

Chapter 13 — Reproducibility and open science

Chapters 8 and 12 covered operational documentation, versioning, storage, and lineage

Learning objectives

- By the end of this part

Reproducibility versus replicability

- Reproducibility and replicability are often used interchangeably
- Reproducibility is obtaining the same results from the same dataset when the same
- Often with a new dataset or under different conditions
- Both matter: reproducibility checks that a published computational path is faithful

Example 13.1 — Same Data and Methods Yield Identical Results

- Example 13.1 — hands-on module
- Example 13.1 illustrates methods reproducibility
- If a researcher publishes a study using a dataset and a specific analytical technique
- That identity is a measure of transparency and robustness in the scientific process
- View files: ``modules/chapter13/example1/``

Example 13.1 — listing

```
"""Example 13.1 – same data and methods yield identical results.
```

```
Demonstrates computational (methods) reproducibility with the standard  
library:
```

```
a seeded pipeline produces a byte-identical result across runs, while  
changing
```

```
the seed or a step breaks that guarantee.
```

```
"""
```

```
from __future__ import annotations
```

```
import hashlib
```

```
import random
```

```
import statistics
```

```
def analysis(seed: int, transform: str = "zscore") -> str:
```

Example 13.2 — Replication with New Climate Data

- Example 13.2 — hands-on module
- Example 13.2 contrasts replicability
- A climate model may be replicated by a different team using a new set of environmental
- Replicability can thus be seen as an extension of reproducibility
- Explore the chapter example module
- View files: ``modules/chapter13/example2/``

Core principles of open science

- Open science seeks to make research and data publicly accessible, transparent
- Accessibility means research outputs, datasets, software
- Transparency means making processes visible
- Collaboration means sharing knowledge and resources across disciplines and institutions so

Example 13.3 — OSF and Zenodo for Open Sharing

- Example 13.3 — hands-on module
- Example 13.3 names widely used platforms for open sharing
- Preprints with the global community
- Accessibility is not only about publishing a paper
- Explore the chapter example module
- View files: ``modules/chapter13/example3/``

Example 13.4 — Publishing Scripts and Notebooks

- Example 13.4 — hands-on module
- Example 13.4 shows transparency through released analysis assets
- Researchers may release the scripts or notebooks used to process their data so others can
- Methodological limitations
- Explore the chapter example module
- View files: ``modules/chapter13/example4/``

Takeaways

- Reproducibility checks the same data and methods
- Open science rests on accessibility, transparency, and collaboration
- Repositories such as the Open Science Framework and Zenodo support open sharing of data
- Publishing scripts and notebooks makes workflows checkable by others
- This chapter's focus is research-facing practice

Next

- Complete the quiz for this part
- The next part covers barriers that still block reproducible work

Learning objectives

- By the end of this part

Three recurring barriers

- Achieving reproducibility in practice remains difficult
- Three barriers recur across fields
- Datasets may be unavailable because of privacy
- Methods may be published without enough detail to rerun the analysis
- Without shared conventions for documentation, versioning

Example 13.5 — HIPAA Limits on Clinical Data Sharing

- Example 13.5 — hands-on module
- Example 13.5 shows how privacy law can block open sharing of clinical data
- Clinical research often involves sensitive patient data that cannot be shared openly under
- Open science therefore pairs sharing goals with de-identification and anonymization so
- Explore the chapter example module
- View files: ``modules/chapter13/example5/``

Example 13.6 — Missing Hyperparameters Block Replication

- Example 13.6 — hands-on module
- Example 13.6 shows how undocumented methods prevent replication of a machine learning
- Making it impossible for others to replicate the experiment
- Either way, the computational path cannot be checked
- Explore the chapter example module
- View files: ``modules/chapter13/example6/``

Example 13.7 — Incompatible Bioinformatics Naming Standards

- Example 13.7 — hands-on module
- Example 13.7 shows how missing community standards hinder cross-lab comparison
- In bioinformatics
- Not optional polish
- Explore the chapter example module
- View files: ``modules/chapter13/example7/``

Example 13.8 — Drug Decisions Depend on Reproducible Trials

- Example 13.8 — hands-on module
- Example 13.8 ties reproducibility standards to high-stakes decisions
- A pharmaceutical company may base drug development on clinical trial results
- They reinforce research integrity when findings affect policy and public health
- Explore the chapter example module
- View files: ``modules/chapter13/example8/``

Foundations bridge to documentation

- The foundation of reproducible research still lies in proper dataset documentation and
- Chapter 8 develops templates, metadata, data dictionaries, provenance records
- The next parts therefore do not repeat that operational material

Takeaways

- Inaccessible data, undocumented methods, and missing standards are recurring blockers
- Privacy regimes such as HIPAA require safe release practices rather than unrestricted
- Missing hyperparameters and incompatible naming prevent faithful reruns and cross-study
- Reproducibility standards protect trust in high-stakes settings such as drug decisions
- Documentation and versioning remain the bridge from Chapters 8 and 12 into research-facing

Next

- Complete the quiz for this part
- The next part covers research documentation, repository metadata for discovery

Learning objectives

- By the end of this part

Research documentation demands

- Beyond the documentation components listed in Chapter 8
- The goal is a checkable published path, not only an internal ops record

Example 13.9 — Undocumented Methodology Blocks Replication

- Example 13.9 — hands-on module
- Example 13.9 shows the cost of unclear methods
- In a medical study
- Surveys of researchers have long found that inadequate documentation of methods and data
- Explore the chapter example module
- View files: ``modules/chapter13/example9/``

Example 13.10 — Repository Metadata Enables Discovery

- Example 13.10 — hands-on module
- Metadata types and roles are covered in Chapter 8
- What matters for open science is publication-grade metadata
- License, so related searches can discover it beyond the original study
- Explore the chapter example module
- View files: ``modules/chapter13/example10/``

Example 13.11 — Unpinned Dataset Versions Break Replication

- Example 13.11 — hands-on module
- Versioning tools are developed in Chapters 8 and 12
- The exact dataset version used in a published analysis must be identifiable and
- Example 13.11 shows what breaks without pinned versions
- Explore the chapter example module
- View files: ``modules/chapter13/example11/``

Takeaways

- Undocumented methods leave published findings checkable only in name
- Publication-grade metadata turns repositories into discovery systems, not dump sites
- Pinning the exact dataset version used in a paper is required for faithful replication
- Chapter 8 and Chapter 12 supply the operational toolkit

Next

- Complete the quiz for this part
- The next part introduces FAIR principles, Findable, Accessible, Interoperable

Learning objectives

- By the end of this part, learners should be able to name the four FAIR principles

What FAIR means

- The FAIR principles, Findable, Accessible, Interoperable
- Findable means datasets can be located through identifiers and rich metadata
- Accessible means retrieval under clear, standardized terms, which may include restrictions
- Interoperable means formats and vocabularies support combination with other datasets and
- Reusable means documentation, metadata

Example 13.12 — Indexing Genomics Data in Public Archives

- Example 13.12 — hands-on module
- Example 13.12 illustrates Findable sharing
- When a cancer genomics dataset is published
- Researchers can then locate it through keywords, metadata, or other relevant fields
- Indexing is what turns a deposited file into a discoverable research asset
- View files: ``modules/chapter13/example12/``

Example 13.13 — Zenodo and Figshare with Access Controls

- Example 13.13 — hands-on module
- Accessible does not mean every dataset must be free of restrictions
- Data must be available under clear terms
- Example 13.13 shows datasets hosted on Zenodo or Figshare
- Explore the chapter example module
- View files: ``modules/chapter13/example13/``

Example 13.14 — FASTQ VCF and Controlled Vocabularies

- Example 13.14 — hands-on module
- Interoperable data is structured for integration with other datasets and analytical tools
- Example 13.14 shows genomic data formatted in standard file types such as FASTQ or VCF so
- Controlled vocabularies such as Gene Ontology or the Unified Medical Language System
- Explore the chapter example module
- View files: ``modules/chapter13/example14/``

Example 13.15 — Community File Formats for Reuse

- Example 13.15 — hands-on module
- Reusable data is accompanied by documentation, metadata
- Example 13.15 prefers community formats such as CSV or JSON for tabular data
- Choosing formats the community already understands is a practical Reusable choice
- Explore the chapter example module
- View files: ``modules/chapter13/example15/``

Takeaways

- FAIR means Findable, Accessible, Interoperable, and Reusable for people and machines
- Public-archive indexing makes genomics and other deposits discoverable
- Accessibility allows clear terms and access controls, not only unrestricted dumps
- Standard formats and controlled vocabularies support interoperability
- Community file formats and licensing documentation enable reuse beyond the original study

Next

- Complete the quiz for this part
- The next part covers data-quality standards for reproducibility and research-facing

Learning objectives

- Context; and describe how tracking updates preserves integrity across dataset versions

Quality standards for reproducibility

- Data quality dimensions and cleaning procedures are treated in earlier chapters
- For reproducibility
- Completeness, consistency, and accuracy support that goal
- Not silent omission

Example 13.18 — Documenting Clinical Trial Dropouts

- Example 13.18 — hands-on module
- Example 13.18 shows completeness documentation for missing participants
- In clinical trials
- Future users need those facts to interpret results and to repeat the analysis under
- Explore the chapter example module
- View files: ``modules/chapter13/example18/``

Provenance as research evidence

- Data provenance is the documentation and tracking of origin and transformations throughout
- It records where data came from, how they were collected, what processes were applied
- Provenance provides transparency and accountability so researchers can verify credibility
- The emphasis here is evidence for reproducible research and open science

Example 13.24 — Genomics Provenance for Sequencing Pipelines

- Example 13.24 — hands-on module
- Example 13.24 shows the provenance fields a genomics study should record
- Provenance should capture the sequencing technology, preprocessing steps
- Without such documentation
- Explore the chapter example module
- View files: ``modules/chapter13/example24/``

Example 13.25 — Tracking Dataset Updates Over Time

- Example 13.25 — hands-on module
- Provenance also improves traceability when datasets change
- Example 13.25 shows that if a dataset undergoes modifications or updates
- In regulated fields such as healthcare or finance
- Explore the chapter example module
- View files: ``modules/chapter13/example25/``

Takeaways

- Quality decisions must be documented if published analyses are to be repeated faithfully
- Completeness includes recording dropouts and other missingness with analytical impact
- Provenance records origin, transforms, context
- Tracking updates links versions so integrity and audits remain possible
- Distinguish Chapter 12 platform lineage from this chapter's research-facing provenance

Next

- Complete the quiz for this part
- The next part covers open-data licensing, balancing openness with privacy

Learning objectives

- By the end of this part

Open license menu

- Creative Commons covers CC0, CC BY, CC BY-SA, CC BY-NC, and CC BY-NC-SA
- Open Data Commons adds database-oriented paths: ODC-By, ODbL, and PDDL
- Companion code beside a dataset usually needs a software license such as MIT
- License choice follows the intended reuse, not the row layout of a CSV

Choosing from the use case

- Ask whether the release is sensitive or community-controlled
- Public-domain paths suit government tables
- Practice these paths in the course license chooser

Licenses and the privacy trade-off

- Responsible open science depends on clear licenses and ethical constraints
- Some teams draft custom or restricted licenses that limit redistribution
- Privacy principles and techniques are developed in Chapter 3
- An unrestricted open license signals broad reuse and is appropriate only after privacy

Example 13.26 — Restricted Licenses for Health Data

- Example 13.26 — hands-on module
- Example 13.26 illustrates restricted licensing for sensitive health data
- Use may be limited to qualified researchers or specific contexts such as academic or
- Even de-identified data may remain re-identifiable when combined with other releases
- Explore the chapter example module
- View files: ``modules/chapter13/example26/``

Ethical obligations after sharing

- Ethical principles for collection and privacy are covered in Chapter 3
- Open science adds obligations that arise from making data public
- Researchers should document scope and limitations
- Consent obtained during collection must also cover the intended public release

Example 13.27 — Avoid Reinforcing Bias with Open Data

- Example 13.27 — hands-on module
- Ethical concerns do not end when data are shared; they extend to reuse
- Example 13.27 warns against discriminatory reuse of hiring or healthcare datasets
- Such data should not be used to reinforce biases or perpetuate systemic inequalities
- If data reveal disparities
- View files: ``modules/chapter13/example27/``

Example 13.28 — Indigenous Control Over Shared Data

- Example 13.28 — hands-on module
- Open science must respect the rights and wishes of people and communities whose data are
- Communities involved in collection
- Example 13.28 emphasizes that Indigenous communities involved in ecological or
- Explore the chapter example module
- View files: ``modules/chapter13/example28/``

Takeaways

- Match the license family to the artifact
- Restricted access, not an open deed, for sensitive health or community-controlled data
- License choice must follow a privacy assessment, not precede it
- Open release creates ongoing ethical duties around bias, security, attribution, and scope
- Including Indigenous communities, retain legitimate control over sharing decisions

Next

- Complete the quiz for this part
- The next part covers reproducible research workflows

Learning objectives

- By the end of this part

Workflow automation

- Containers so analyses can be independently verified
- Automation tools such as Make
- Consistency also requires a stable environment

Example 13.29 — Automated Bioinformatics Workflow File

- Example 13.29 — hands-on module
- Example 13.29 captures a multi-step bioinformatics pipeline in a workflow definition
- A typical pipeline might involve data preprocessing, alignment, and statistical analysis
- With automation tools
- Explore the chapter example module
- View files: ``modules/chapter13/example29/``

Example 13.31 — Notebook Pipeline from Load to Plots

- Example 13.31 — hands-on module
- Jupyter notebooks integrate code, data, and documentation in a single document
- Example 13.31 sketches a typical analysis narrative
- Interactivity and sharing
- Explore the chapter example module
- View files: ``modules/chapter13/example31/``

Example 13.34 — Docker Image for Sequence Alignment Tools

- Example 13.34 — hands-on module
- Reproducibility depends on a stable computational environment
- Docker packages code, dependencies
- Example 13.34 pins bioinformatics tool versions inside a Docker image
- Explore the chapter example module
- View files: ``modules/chapter13/example34/``

Example 13.36 — Docker Image for ML Training Stack

- Example 13.36 — hands-on module
- The same container pattern applies to machine learning
- Example 13.36 freezes Python library versions for model training
- Shared images reduce “it works on my machine” failures across local, cluster
- Explore the chapter example module
- View files: ``modules/chapter13/example36/``

Takeaways

- Workflow files encode ordered
- Notebooks make methods and results literate and shareable
- Docker pins tools and libraries so bioinformatics and machine-learning stacks travel with
- Containers and workflow definitions close the execution gap left by licenses and prose

Next

- Complete the quiz for this part
- Model regressions before results are published

Learning objectives

- By the end of this part

CI for reproducible research

- For research workflows, tests and validates changes to code or data pipelines
- Services such as GitHub Actions, GitLab CI
- The goal is to catch silent breaks early

Example 13.37 — CI Checks Preprocessing and Inputs

- Example 13.37 — hands-on module
- Example 13.37 uses continuous integration to validate pipeline prerequisites
- Automated checks can verify that preprocessing steps run and that required inputs are
- Catching input and preprocessing failures in CI prevents wasted compute and unreproducible
- Explore the chapter example module
- View files: ``modules/chapter13/example37/``

Example 13.38 — Tests for Cross-Environment Consistency

- Example 13.38 — hands-on module
- Environment drift is a common reproducibility failure
- Example 13.38 emphasizes tests for cross-environment consistency
- Automated tests that compare outputs or checksums across environments expose dependency
- Explore the chapter example module
- View files: ``modules/chapter13/example38/``

Example 13.39 — CI Retests Model Training After Updates

- Example 13.39 — hands-on module
- Machine-learning pipelines change when data, features, or training code update
- Example 13.39 shows CI retesting model training after such updates so regressions in
- Continuous checks keep the published training path aligned with the current repository
- Explore the chapter example module
- View files: ``modules/chapter13/example39/``

Automated workflow documentation

- Automation also supports workflow documentation
- The published narrative stays synchronized with the executable path
- Combined with CI

Takeaways

- Continuous integration turns reproducibility from a manual hope into a repeated check
- CI can validate preprocessing and inputs before costly downstream stages
- Cross-environment tests catch dependency drift across machines and containers
- Retrain CI detects model regressions when data or code change
- Automated documentation keeps published workflows aligned with what actually executed

Next

- Complete the quiz for this part
- The next part covers R Markdown reports and DVC for tracking training data, model weights

Learning objectives

- By the end of this part

Literate reporting with R Markdown

- R Markdown creates dynamic, reproducible reports that integrate code, data
- When the document is compiled, embedded code executes and tables, figures
- That coupling keeps analysis, narrative
- Outputs can include HTML, PDF, or Word for flexible sharing

Example 13.41 — RMarkdown Public Health Analysis Report

- Example 13.41 — hands-on module
- Example 13.41 regenerates an analysis report from an R Markdown source
- Generate visualizations in one document
- As the report compiles
- Explore the chapter example module
- View files: ``modules/chapter13/example41/``

DVC for large research artifacts

- Large datasets in genomics
- DVC versions large datasets and models so they remain reproducible and shareable
- Every modification remains traceable

Example 13.44 — DVC Tracks Training Data and Weights

- Example 13.44 — hands-on module
- Example 13.44 versions training data, model weights, and evaluation outputs with DVC
- In a machine learning project
- Without that linkage
- Explore the chapter example module
- View files: ``modules/chapter13/example44/``

Example 13.45 — DVC Pipeline for Preprocess Train Evaluate

- Example 13.45 — hands-on module
- DVC also supports data pipelines that automate flow through the research process
- Example 13.45 defines a versioned pipeline across preprocessing, model training
- Each step's inputs and outputs are tracked so regenerating results is a single pipeline
- Explore the chapter example module
- View files: ``modules/chapter13/example45/``

Takeaways

- R Markdown couples narrative and executable analysis so published reports recompile
- DVC keeps large training data and model weights retrievable alongside code
- Versioned DVC pipelines encode preprocess, train
- Literate reports plus data versioning close the gap between papers and the artifacts they

Next

- Complete the quiz for this part
- A brief climate-modeling outlook

Learning objectives

- By the end of this part

Cloud collaboration for scale

- Distributed teams need shared infrastructure for large datasets and scalable computation
- Cloud platforms such as Amazon Web Services, Google Cloud
- Chapter 12 covers storage architectures and platform operations

Example 13.47 — Cloud VMs for Consistent Workflows

- Example 13.47 — hands-on module
- Example 13.47 provisions cloud virtual machines so collaborators share the same runtime
- Cloud-based VMs can be configured to run specific research workflows
- Data move between local machines, clusters, and the cloud
- Explore the chapter example module
- View files: ``modules/chapter13/example47/``

Example 13.50 — Environment Drift Breaks Identical Code

- Example 13.50 — hands-on module
- Even with mature tools, full reproducibility often fails
- Example 13.50 shows environment drift as a common pitfall
- Containerization tools such as Docker or virtual environments such as conda encapsulate
- Explore the chapter example module
- View files: ``modules/chapter13/example50/``

Example 13.55 — DVC Links Retraining to Data Versions

- Example 13.55 — hands-on module
- Maintaining audit trails is essential for checkable research
- Just as code is versioned with Git
- Example 13.55 shows DVC linking retraining runs to data versions
- Explore the chapter example module
- View files: ``modules/chapter13/example55/``

Climate case study and future outlook

- Large collaborative programs such as CMIP6 illustrate why these practices matter at
- Ethical licensing as research becomes more computational and collaborative
- The chapter's tools are not optional polish

Takeaways

- Cloud VMs and containers help teams share consistent runtimes at scale
- Environment drift remains a top failure mode when dependencies are unpinned
- Audit trails that link retraining to data versions make published models checkable
- Climate-scale collaborations show why FAIR, versioning
- The chapter closes by tying research-facing reproducibility to the operational foundations

Next

- Complete the quiz for this part
- Complete the quiz for this part to finish Chapter 13